
Basic to Intermediate

1. What is the role of an SRE in an organization?

An SRE (Site Reliability Engineer) is responsible for ensuring that applications and infrastructure are reliable, scalable, secure, and highly available in production environments. The primary goal of an SRE is to reduce operational issues through automation, monitoring, observability, and proactive incident management.

Key responsibilities include:

- Monitoring system health and application performance.
- Managing incident response and root cause analysis.
- Automating repetitive operational tasks.
- Implementing CI/CD and Infrastructure as Code practices.
- Improving availability, scalability, and fault tolerance.
- Defining and tracking SLAs, SLOs, and SLIs.
- Collaborating with development and operations teams to improve production stability.

2. How is SRE different from DevOps?

DevOps is a culture and methodology focused on collaboration between development and operations teams, while SRE is an engineering implementation of DevOps principles using software engineering practices.

DevOps	SRE
Focuses on collaboration and faster delivery	Focuses on reliability and operational excellence
Cultural approach	Engineering discipline
Encourages automation	Implements automation with measurable reliability goals
Shared ownership model	Uses SLOs, SLIs, and error budgets
General methodology	Specific operational framework

In short:

- DevOps answers: “How do we deliver software faster?”
- SRE answers: “How do we keep systems reliable while delivering faster?”

3. What is SLA, SLO, and SLI?

SLA (Service Level Agreement): An SLA is a formal agreement between the service provider and customer that defines expected service availability and penalties if commitments are not met.

Example:

- 99.9% uptime guarantee per month.

SLO (Service Level Objective): An SLO is the internal reliability target set for a service.

Example:

- API response success rate should be 99.95%.

SLI (Service Level Indicator): An SLI is the actual measurable metric used to evaluate system performance.

Examples:

- Request latency
- Error rate
- Availability percentage
- Response time

Term	Example
SLI	API success rate = 99.92%
SLO	Target success rate = 99.9%
SLA	Customer contract promises 99.5% uptime

4. How do you handle production incidents?

I follow a structured incident management process:

Step 1: Detection

- Identify alerts from monitoring tools like Prometheus, Grafana, Datadog, or CloudWatch.

Step 2: Triage

- Assess severity, business impact, and affected services.
- Categorize incident priority (Sev-1, Sev-2, etc.).

Step 3: Communication

- Inform stakeholders through Slack, Teams, PagerDuty, or incident channels.
- Maintain regular updates.

Step 4: Mitigation

- Rollback deployments if needed.
- Restart failed services.
- Scale infrastructure.
- Redirect traffic or activate failover.

Step 5: Root Cause Analysis

- Analyze logs, metrics, traces, and deployment history.

Step 6: Postmortem

- Document RCA.
- Identify preventive actions.
- Improve monitoring and automation.

5. What steps do you take during a Sev-1 outage?

A Sev-1 outage is a critical production incident affecting major business functionality.

My approach:

1. Immediately acknowledge the alert.
2. Create an incident bridge/call.
3. Engage relevant teams quickly.
4. Identify blast radius and affected systems.
5. Stabilize the environment first.
6. Roll back recent deployments if necessary.
7. Check infrastructure health:
 - CPU
 - Memory
 - Disk
 - Network
 - Kubernetes pods
 - Database connections
8. Review logs and monitoring dashboards.
9. Communicate status updates regularly.

10. Restore services with minimum downtime.

11. Conduct detailed RCA after recovery.

Main priority during Sev-1:

- Restore service first.
- Investigate deeply later.

6. Explain error budgets.

An error budget represents the amount of acceptable failure or downtime allowed within an SLO target.

Example:

If the SLO is 99.9% uptime per month:

$$99.9\% \text{ availability} = 0.1\% \text{ allowed downtime}$$

This means:

- Around 43 minutes of downtime is acceptable monthly.

Importance of Error Budgets

- Balances innovation and reliability.
- Allows teams to release features faster while controlling risk.
- If error budget is exhausted:
 - Stop risky deployments.
 - Focus on reliability improvements.

7. How do you improve system reliability?

I improve system reliability through multiple engineering practices:

Monitoring & Observability

- Implement metrics, logs, tracing, and alerts.

Automation

- Automate deployments, scaling, backups, and recovery.

High Availability

- Use load balancers, multiple availability zones, and redundancy.

Infrastructure as Code

- Use Terraform, CloudFormation, or Ansible for consistency.

CI/CD Validation

- Add automated testing and deployment validation.

Capacity Planning

- Monitor resource utilization and scale proactively.

Incident Reviews

- Conduct blameless postmortems and implement preventive fixes.

Chaos Testing

- Test failure scenarios to validate resilience.

8. What is MTTR and how do you reduce it?

MTTR stands for Mean Time To Recovery (or Repair).

It measures how quickly systems recover after failures.

Formula:

$$MTTR = \frac{\text{Total Recovery Time}}{\text{Number of Incidents}}$$

Ways to reduce MTTR:

- Better monitoring and alerting.
- Automated rollback mechanisms.
- Runbooks and SOP documentation.
- Faster incident escalation.
- Centralized logging.
- Kubernetes self-healing features.
- Auto-scaling and failover strategies.
- Regular disaster recovery drills.

Lower MTTR means faster restoration and improved reliability.

9. What is high availability?

High Availability (HA) means designing systems to remain operational with minimal downtime even during failures.

Common HA strategies:

- Multi-zone deployment
- Load balancing
- Database replication
- Auto-scaling

- Redundant servers
- Kubernetes replica sets
- Failover clusters

Example:

If one application server fails, traffic automatically redirects to healthy instances without impacting users.

Typical HA targets:

- 99.9%
- 99.99%
- 99.999% (“Five Nines”)

10. Explain disaster recovery planning.

Disaster Recovery (DR) planning is the strategy used to restore systems, applications, and data after major failures such as:

- Data center outage
- Cloud region failure
- Cyberattack
- Database corruption
- Natural disaster

Important DR components:

Backup Strategy

- Regular database and infrastructure backups.

Replication

- Cross-region or multi-region replication.

Recovery Objectives

RPO (Recovery Point Objective)

Maximum acceptable data loss.

RTO (Recovery Time Objective)

Maximum acceptable recovery time.

Failover Mechanism

- Switch traffic to secondary environments.

DR Testing

- Conduct periodic failover drills.

Documentation

- Maintain detailed recovery procedures and contacts.

Example:

In AWS:

- Use Multi-AZ databases.
- Replicate backups to another region.
- Use Route53 failover routing.
- Maintain standby infrastructure using Terraform.

Scenario-Based Questions

1. A production API latency suddenly increases. How would you investigate?

I would follow a systematic troubleshooting approach to identify whether the issue is coming from the application layer, infrastructure, database, or external dependencies.

Step 1: Verify the Alert

- Check monitoring dashboards in Grafana, Datadog, Prometheus, or CloudWatch.
- Identify:
 - Response time increase
 - Error rates
 - Traffic spikes
 - Affected endpoints

Step 2: Check Infrastructure Metrics

- Analyze:
 - CPU usage
 - Memory consumption
 - Disk I/O
 - Network latency
 - Pod/container health

Step 3: Review Application Logs

- Search for:
 - Exceptions

- Timeout errors
- Slow queries
- Connection failures

Step 4: Check Recent Changes

- Verify:
 - Recent deployments
 - Infrastructure modifications
 - Database schema changes
 - Configuration updates

Step 5: Analyze Dependencies

- Check:
 - Database performance
 - External APIs
 - Cache systems like Redis
 - Message queues

Step 6: Use Distributed Tracing

- Use tools like Jaeger or OpenTelemetry to identify slow service calls.

Step 7: Mitigation

- Scale services horizontally.
- Roll back problematic deployments.
- Optimize slow database queries.
- Enable caching if required.

The primary goal is to reduce user impact first and then perform detailed root cause analysis.

2. CPU utilization is constantly above 90% on critical servers. What would you do?

High CPU utilization indicates resource exhaustion or inefficient workload handling.

Investigation Steps

Step 1: Identify High CPU Processes

Commands:

top

htop


```
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu
```

Step 2: Check Application Behavior

- Identify:
 - Infinite loops
 - High traffic spikes
 - Memory leaks
 - Background jobs
 - Thread contention

Step 3: Review Monitoring Trends

- Analyze historical CPU patterns.
- Determine whether it is:
 - Sudden spike
 - Gradual increase
 - Peak-hour issue

Step 4: Check Infrastructure Scaling

- Verify:
 - Auto-scaling configuration
 - Kubernetes HPA settings
 - Load balancing distribution

Step 5: Immediate Mitigation

- Scale vertically or horizontally.
- Restart unhealthy services if necessary.
- Redistribute traffic.
- Reduce unnecessary workloads.

Step 6: Long-Term Fix

- Optimize application code.
- Tune JVM or runtime parameters.
- Implement caching.
- Optimize database calls.

3. One microservice keeps restarting in production. How do you troubleshoot?

Repeated restarts usually indicate application crashes, resource limits, or configuration issues.

Step 1: Check Pod or Container Status

```
kubectl get pods
```

```
kubectl describe pod <pod-name>
```

Step 2: Analyze Logs

```
kubectl logs <pod-name> --previous
```

Look for:

- Application exceptions
- OOMKilled errors
- Dependency failures
- Configuration issues

Step 3: Check Resource Limits

Verify:

- CPU limits
- Memory limits
- Disk space

Step 4: Validate Health Checks

Review:

- Liveness probe
- Readiness probe
- Startup probe

Incorrect probes can cause restart loops.

Step 5: Check External Dependencies

- Database connectivity
- API availability
- DNS resolution
- Secrets/configmaps

Step 6: Review Recent Changes

- Deployment updates
- Image versions
- Environment variables

Step 7: Mitigation

- Roll back deployment if needed.
- Increase memory temporarily.
- Fix application exceptions.

4. Users report intermittent downtime but monitoring shows everything healthy. What next?

This situation usually indicates gaps in observability or issues outside infrastructure metrics.

Investigation Approach

Step 1: Validate User Reports

- Identify:
 - Specific regions
 - Browsers
 - Devices
 - Time windows

Step 2: Check Synthetic Monitoring

- Use uptime checks from multiple regions.

Step 3: Analyze Network Layer

- DNS issues
- CDN problems
- Load balancer intermittency
- Firewall rules

Step 4: Check Application-Level Metrics

Sometimes infrastructure appears healthy while:

- APIs fail intermittently
- Thread pools exhaust
- Database latency spikes

Step 5: Review Logs and Traces

- Search for transient failures.
- Correlate timestamps with user complaints.

Step 6: Validate Third-Party Dependencies

- External APIs
- Payment gateways

- Identity providers

Step 7: Improve Observability

Implement:

- Distributed tracing
- Real user monitoring (RUM)
- Better alerting thresholds

5. A deployment caused service degradation. How would you roll back safely?

Step 1: Identify Impact

- Verify:
 - Error rates
 - Latency
 - User impact
 - Affected services

Step 2: Stop Further Deployment

- Pause CI/CD pipeline immediately.

Step 3: Roll Back Safely

In Kubernetes:

```
kubectl rollout undo deployment <deployment-name>
```

Or:

- Revert Helm release
- Restore previous container image

Step 4: Validate Recovery

Check:

- Application health
- Metrics
- Logs
- Traffic flow

Step 5: Communicate Status

- Inform stakeholders about rollback and recovery progress.

Step 6: Root Cause Analysis

Investigate:

- Code changes
- Configuration errors
- Database migrations
- Dependency incompatibility

Step 7: Prevent Recurrence

Implement:

- Canary deployments
- Blue-green deployment
- Automated testing
- Feature flags

6. Database connections are exhausted. How do you identify the root cause?

Step 1: Check Active Connections

For PostgreSQL:

```
SELECT * FROM pg_stat_activity;
```

Step 2: Identify Connection Sources

- Application servers
- Background jobs
- Reporting services

Step 3: Check Connection Pooling

Verify:

- Pool size
- Idle connections
- Connection leaks

Step 4: Analyze Long-Running Queries

Identify:

- Slow queries
- Locked transactions
- Deadlocks

Step 5: Review Recent Deployments

- New code may not close connections properly.

Step 6: Immediate Mitigation

- Restart leaking services if required.
- Increase connection limits temporarily.
- Kill stale sessions.

Step 7: Long-Term Improvements

- Use connection pooling.
- Optimize queries.
- Implement timeout policies.
- Monitor DB utilization continuously.

7. How do you manage incident communication during outages?

Effective communication is critical during outages.

My approach:

Step 1: Create Incident Channel

- Slack bridge
- Teams call
- War room

Step 2: Assign Roles

- Incident Commander
- Communication Lead
- Technical Responders

Step 3: Provide Regular Updates

Include:

- Current impact
- Services affected
- Mitigation progress
- ETA if available

Step 4: Avoid Assumptions

Only communicate verified information.

Step 5: Maintain Stakeholder Visibility

Update:

- Management

- Support teams
- Customers if required

Step 6: Post-Incident Summary

Share:

- Root cause
- Resolution
- Preventive measures

Clear communication reduces confusion and improves coordination.

8. How would you reduce alert fatigue?

Alert fatigue happens when teams receive too many low-value alerts.

Reduction Strategies

Prioritize Alerts

Classify:

- Critical
- Warning
- Informational

Remove Noisy Alerts

- Eliminate duplicate or non-actionable alerts.

Improve Alert Thresholds

- Avoid overly sensitive triggers.

Use Alert Correlation

- Group related alerts together.

Implement SLO-Based Alerting

Alert only when customer impact exists.

Use Smart Routing

Send alerts to correct teams only.

Automate Recovery

- Self-healing scripts
- Auto-remediation workflows

Regular Alert Review

Continuously audit alert quality and usefulness.

9. How do you ensure reliability in multi-region deployments?

Multi-region architecture improves availability and disaster recovery.

Key Practices

Deploy Across Multiple Regions

- Active-active or active-passive architecture.

Data Replication

- Cross-region database replication.

Global Load Balancing

Use:

- Route53
- Azure Traffic Manager
- GCP Load Balancer

Health Checks and Failover

Automatically redirect traffic during failures.

Infrastructure as Code

Ensure identical infrastructure across regions.

Observability

Monitor:

- Replication lag
- Regional latency
- Service health

Disaster Recovery Drills

Regularly test failover scenarios.

Minimize Regional Dependencies

Avoid single-region bottlenecks.

10. How do you handle postmortem analysis?

A postmortem is conducted after incidents to identify root causes and prevent recurrence.

My approach:

Step 1: Gather Timeline

Document:

- Incident start
- Detection time
- Mitigation actions
- Recovery time

Step 2: Identify Root Cause

Analyze:

- Logs
- Metrics
- Deployment history
- Infrastructure changes

Step 3: Focus on Process Improvement

Blameless culture is important.

Step 4: Document Impact

Include:

- Downtime duration
- Users affected
- Revenue or business impact

Step 5: Define Action Items

Examples:

- Better monitoring
- Automation improvements
- Runbook updates
- Capacity upgrades

Step 6: Track Preventive Actions

Ensure remediation tasks are completed.

A good postmortem converts incidents into long-term reliability improvements.

Linux Administration

1. Explain the Linux boot process.

DevOps Interview Guide- DevOps Diaries

The Linux boot process is the sequence of steps the system follows from powering on the server until the operating system becomes fully operational.

Boot Process Stages

Step 1: BIOS/UEFI Initialization

- The BIOS or UEFI firmware starts after the system powers on.
 - It performs hardware checks called POST (Power-On Self-Test).
 - It identifies bootable devices like HDD, SSD, or network boot.
-

Step 2: Bootloader (GRUB)

- GRUB (Grand Unified Bootloader) loads the Linux kernel into memory.
- It provides boot menu options.
- Kernel parameters can also be modified here.

GRUB configuration location:

`/etc/default/grub`

Step 3: Kernel Initialization

- The Linux kernel decompresses and initializes.
- It detects hardware drivers.
- It mounts the root filesystem.

Kernel file usually:

`/boot/vmlinuz`

Step 4: Init/Systemd Process

- Kernel starts the first process: PID 1
- Modern Linux systems use: system
- Older systems used: init

Step 5: System Services Start

- Services like networking, SSH, cron, and logging start.

Examples:

`systemctl status sshd`

`systemctl status crond`

Step 6: User Login

- Login prompt or graphical interface becomes available.

2. Difference between soft link and hard link.

Hard Link	Soft Link (Symbolic Link)
Direct reference to inode	Pointer to file path
Cannot link directories normally	Can link directories
Works even if original file renamed	Breaks if original deleted
Shares same inode number	Has different inode
Cannot cross filesystem	Can cross filesystem

Create Hard Link: `ln file1 hardlink`

Create Soft Link: `ln -s file1 softlink`

Check Inode: `ls -li`

3. What happens when you run a command in Linux?

When a command is executed, Linux performs multiple internal operations.

Step-by-Step Flow

Step 1: Shell Accepts Command

`ls`

`pwd`

Shell can be:

- `bash`
- `zsh`
- `sh`

Step 2: Shell Searches PATH

Linux checks command location using: `echo $PATH`

Common directories: `/bin`

`/usr/bin`

Step 3: Fork System Call

Shell creates a child process using: `fork()`

Step 4: Exec System Call

Child process loads the command into memory using: `exec()`

Step 5: Kernel Executes Command

Kernel allocates:

- CPU
- Memory
- File descriptors

Step 6: Output Returned

The command output is displayed in terminal.

4. How do you check memory usage?

Basic Memory Usage: `free -h`

Detailed Process Memory Usage: `top` or `htop`

Check Specific Process Memory: `ps aux --sort=-%mem`

Virtual Memory Statistics: `vmstat`

Memory Information File: `cat /proc/meminfo`

5. Explain process states in Linux.

Every process in Linux exists in a specific state

State	Meaning
R	Running or runnable
S	Sleeping
D	Uninterruptible sleep
T	Stopped
Z	Zombie
X	Dead

Check Process States: ps aux or top

Important States

Running (R): Process actively using CPU.

Sleeping (S): Waiting for event or input.

Uninterruptible Sleep (D)

- Waiting for I/O operation.
- Cannot be interrupted easily.

Zombie (Z): Process completed but parent has not collected exit status.

6. What is the difference between cron and systemd timers?

Cron	systemd Timers
Traditional scheduler	Modern scheduler
Uses crontab	Uses timer units
Limited logging	Better logging with journalctl
No dependency management	Supports dependencies
Simpler	More flexible
Less monitoring	Integrated with systemd

7. Explain swap memory.

Swap memory is disk space used as virtual memory when RAM becomes full.

It helps prevent application crashes during memory exhaustion.

Types

- Swap partition
- Swap file

Check Swap Usage : swapon --show

Advantages

- Prevents OOM (Out Of Memory)
- Supports memory overflow handling

Disadvantages

- Disk is slower than RAM

- Excessive swapping reduces performance

8. What is inode in Linux?

An inode is a metadata structure that stores information about a file.

It does NOT store filename directly.

Inode Stores

- File size
- Permissions
- Owner ID
- Group ID
- Timestamps
- Disk block locations

Check Inode Number: `ls -li`

9. How do you find large files in a server?

Find Files Larger Than 1GB: `find / -type f -size +1G`

Find Top 10 Largest Files: `du -ah / | sort -rh | head -10`

Check Directory Usage: `du -sh *`

Interactive Disk Usage Tool: `ncdu`

Common Investigation Areas

- Log files
- Backup directories
- Docker images
- Temp files

10. Difference between ext4 and xfs.

ext4	XFS
Traditional Linux filesystem	High-performance filesystem
Better for smaller files	Better for large files
Faster fsck recovery	Slower repair but scalable
Supports journaling	Supports journaling
Easier resizing	Excellent scalability
Common in Ubuntu	Common in RHEL/CentOS

ext4 Features

- Stable
- Reliable
- Widely used
- Good general-purpose filesystem

XFS Features

- Better for enterprise workloads
- Handles massive filesystems efficiently
- Better parallel I/O performance

Linux Troubleshooting

1. A server becomes unreachable. What steps will you take?

Step-by-Step Troubleshooting Approach

A. Verify Basic Connectivity:

```
ping <server-ip>
traceroute <server-ip>
telnet <server-ip> 22
nc -zv <server-ip> 22
```

Check whether:

Server is completely down
Network path is broken
SSH port is blocked

B. Check Cloud Provider Status

AWS

- Verify EC2 instance state
- Check Security Groups
- Check NACLs
- Verify route tables
- Check Elastic IP association
- Review CloudWatch metrics

Azure

- Verify VM status
- Check NSG rules
- Verify UDR routes
- Check Azure Bastion access
- Review Azure Monitor metrics

C. Access Through Console

If SSH is not working:

- AWS EC2 Serial Console
- Azure Serial Console
- VMware/vCenter Console
- IPMI/iLO/DRAC

D. Verify SSH Service

```
systemctl status sshd
```

```
journalctl -u sshd
```

Restart if needed: `systemctl restart sshd`

E. Check Resource Exhaustion

```
top
```

```
free -m
```

```
df -h
```

```
df -i
```

Possible causes:

- Disk full
- Inode exhaustion

- Memory exhaustion
- CPU saturation

F. Verify Network Configuration

ip addr

ip route

ss -tulpn

iptables -L

firewall-cmd --list-all

G. Review System Logs

journalctl -xe

dmesg

tail -f /var/log/messages

tail -f /var/log/syslog

Look for:

- Kernel panic
- OOM killer
- Network failures
- Filesystem corruption

2. Disk usage suddenly becomes 100%. How do you investigate?

- Step 1: Check Filesystem Usage : df -h
- Step 2: Identify Large Directories: du -sh /* 2>/dev/null | sort -hr
- Step 3: Find Large Files: find / -type f -size +1G 2>/dev/null
- Step 4: Check Deleted Open Files: lsof | grep deleted

Restart the process if required.

- Step 5: Check Inode Usage: df -i

Millions of small files can exhaust inodes.

- Step 6: Investigate Logs:

du -sh /var/log/*

journalctl --disk-usage

- Step 7: Docker/Kubernetes Investigation

docker system df

docker ps -a

3. High load average but low CPU utilization — what does this indicate?

This usually indicates processes are waiting on resources rather than actively consuming CPU.

Most common causes:

A. Disk I/O Bottleneck

Processes waiting for disk.

Check: `iostat -xz 1`

`iotop`

`vmstat 1`

High:

- %wa (I/O wait)
- Disk queue size

B. Uninterruptible Sleep (D State)

Processes blocked on I/O.

Check: `ps aux | awk '$8 ~ /D/'`

C. NFS or Storage Issues

Remote mount latency can increase load.

Check:

`mount`

`nfsstat`

D. Thread Contention / Locking

Applications waiting on locks.

Use: `strace -p <pid>`

E. Network Filesystem Delays

Slow SAN/NAS storage.

4. How do you identify memory leaks?

Step 1: Check Overall Memory Usage

`free -m`

`top`

`htop`

`vmstat 1`

Step 2: Identify Top Memory Consumers

```
ps aux --sort=-%mem | head
```

Step 3: Monitor Process Growth

```
watch -n 5 "ps -o pid,user,%mem,rss,cmd -p <PID>"
```

If memory continuously grows without release → possible leak.

Step 4: Analyze with Tools

Linux Native

```
pmap -x <PID>
```

```
smem
```

Step 5: Check OOM Events

```
dmesg | grep -i oom
```

```
journalctl | grep -i killed
```

5. How would you troubleshoot slow SSH login?

Step 1: Test Connection Timing

```
time ssh user@server
```

```
ssh -vvv user@server
```

Step 2: DNS Reverse Lookup Issues

Very common cause.

Disable DNS lookup in SSH daemon: `vim /etc/ssh/sshd_config`

Add:

```
UseDNS no
```

```
GSSAPIAuthentication no
```

Restart: `systemctl restart sshd`

Step 3: Check Authentication Delays

Possible LDAP/AD delays.

Review: `cat /etc/nsswitch.conf`

Step 4: Verify SSH Logs

```
journalctl -u sshd
```

```
tail -f /var/log/secure
```

Step 5: Check Network Latency

```
ping
```

```
mtr
```

tracert

Step 6: Check Resource Exhaustion

High load or memory pressure can delay SSH.

6. A process is stuck in D state. What does it mean?

D State = Uninterruptible Sleep

The process is waiting for I/O and cannot be interrupted even with SIGKILL.

Usually caused by:

- Disk I/O issues
- NFS hangs
- SAN/storage problems
- Kernel driver issues

Identify D State Processes

```
ps aux | awk '$8 ~ /D/'
```

Investigate Parent Cause

```
strace -p <PID>
```

```
lsof -p <PID>
```

```
cat /proc/<PID>/stack
```

Check Storage Health

```
dmesg
```

```
iostat -xz 1
```

```
multipath -ll
```

NFS Checks

```
mount | grep nfs
```

```
nfsstat
```

7. How do you debug DNS issues in Linux?

Step 1: Verify DNS Resolution

```
nslookup google.com
```

```
dig google.com
```

```
host google.com
```

Step 2: Check Resolver Configuration

cat /etc/resolv.conf

Verify:

- DNS server entries
- Search domains

Step 3: Check DNS Reachability

ping <dns-server>

telnet <dns-server> 53

Step 4: Check Local DNS Cache

Systemd: systemd-resolve --status

Flush cache: resolvectl flush-caches

Step 5: Check NSS Configuration :

cat /etc/nsswitch.conf

Step 6: Packet Capture

tcpdump -i any port 53

8. What logs would you check for application crashes?

Linux System Logs

/var/log/messages

/var/log/secure

Application Logs

Common locations:

/var/log/app/

/opt/app/logs/

Web Server Logs

Nginx: /var/log/nginx/error.log

Apache: /var/log/httpd/error_log

Docker: docker logs <container>

Kubernetes : kubectl logs <pod>

9. How do you troubleshoot network latency?

Step 1: Basic Latency Tests

ping

mtr

tracert

Step 2: Measure Packet Loss: mtr -rw <host>

Step 3: Check Interface Statistics

ip -s link

ethtool eth0

Step 4: Check Bandwidth Usage

iftop

nload

sar -n DEV 1

Step 5: Verify DNS Delays: dig

Nslookup

Step 6: Analyze TCP Connections

ss -s

netstat -an

Step 7: Packet Capture

tcpdump

Wireshark

Step 8: Cloud Investigation

AWS

- Security Groups
- NACLs
- ENI throughput
- VPC Flow Logs
- Transit Gateway latency

Azure

- NSG Flow Logs
- Effective routes
- Network Watcher
- Connection Monitor

10. How do you identify zombie processes?

What is a Zombie Process?

A terminated process whose parent has not yet read its exit status.

State = Z

Identify Zombies: `ps aux | grep Z`

Or

`ps -el | grep Z`

Find Parent Process: `ps -o ppid= -p <zombie_pid>`

Check Parent Status: `ps -fp <PPID>`

Resolution

Option 1: Restart Parent Process: `systemctl restart <service>`

Option 2: Kill Parent Process: `kill -HUP <PPID>`

Option 3: Reboot: If parent is kernel-related or unrecoverable.

Linux Commands

1. top

Purpose

Real-time monitoring of:

- CPU usage
- Memory usage
- Running processes
- Load average

Important Information in top

- CPU Usage
- %us → User CPU
- %sy → System CPU
- %id → Idle
- %wa → I/O wait

Memory:

used

free

buff/cache

swap

Load Average: load average: 1.20, 0.90, 0.70

Represents runnable processes over:

- 1 min
- 5 min
- 15 min

Useful Operations Inside top

Key	Purpose
P	Sort by CPU
M	Sort by Memory
k	Kill process
r	Renice process
c	Full command line

2. vmstat

Purpose

Displays:

- Memory
- CPU
- Swap
- Process
- I/O statistics

Command : vmstat 1

Updates every 1 second.

Column	Meaning
r	Runnable processes
b	Processes in uninterruptible sleep
swpd	Swap used
free	Free memory
si/so	Swap in/out
bi/bo	Block input/output
us	User CPU
sy	System CPU
id	Idle CPU
wa	I/O wait

3. iostat

Purpose

Monitor:

- Disk performance
- CPU utilization
- I/O bottlenecks

Command: `iostat -xz 1`

Field	Meaning
%util	Disk utilization
await	Average wait time
svctm	Service time
r/s	Reads/sec
w/s	Writes/sec

4. sar

Purpose

Historical performance monitoring.

Part of: sysstat

CPU Usage:

Memory: sar -r 1 5

Network: sar -n DEV 1 5

Disk: sar -d 1 5

Benefits

Useful for:

- Historical analysis
- Performance baselining
- Capacity planning

5. lsof

Purpose

Lists open files and sockets.

Linux treats everything as a file.

Find Port Usage: lsof -i :8080

Find Deleted Open Files: lsof | grep deleted

Check Process Open Files: lsof -p <PID>

6. strace

Purpose

Trace system calls made by a process.

Very useful for debugging.

Attach to Process: strace -p <PID>

Trace Command: strace ls

Common Use Cases

- Hung processes

- Permission denied issues
- Missing files
- Network calls
- DNS resolution issues

7. tcpdump

Purpose

Packet capture and network troubleshooting.

Capture Traffic: `tcpdump -i eth0`

Capture Specific Port: `tcpdump -i eth0 port 80`

DNS Traffic: `tcpdump port 53`

Save Capture: `tcpdump -w capture.pcap`

Troubleshooting Use Cases

- Packet drops
- DNS failures
- TCP retransmissions
- SSL issues
- Connectivity problems

8. netstat

Purpose

Network statistics and connections.

Older tool; mostly replaced by ss.

Active Connections

Option	Meaning
t	TCP
u	UDP
l	Listening
n	Numeric
p	Process

9. ss

Purpose

Modern replacement for netstat.

Faster and more efficient.

Listening Ports: ss -tulnp

Established Connections: ss -ant

Socket Statistics: ss -s

Benefits Over netstat

- Faster
- Better scalability
- More accurate on busy systems

10. journalctl

Purpose

View logs from systemd journal.

- View All Logs: journalctl
- Service Logs: journalctl -u nginx
- Real-Time Logs: journalctl -f
- Boot Logs: Boot Logs
- Since Time: journalctl --since "1 hour ago"
- Priority Filtering: journalctl -p err

Q. Difference Between grep, awk, and sed

Tool	Primary Purpose	Best Use Case
grep	Search/filter text	Find matching lines
awk	Pattern scanning + processing	Column-based processing
sed	Stream editor	Modify text

1. grep

Purpose

Search matching patterns.

Best For

- Searching logs
- Filtering output
- Pattern matching

2. awk

Purpose

Text processing and reporting.

Works column-wise.

Best For

- Parsing structured text
- Reports
- CSV/log analysis

3. sed

Purpose

Modify and transform text streams.

Best For

- Text replacement
- File modifications
- Automation scripts

Q. Write a command to find files older than 30 days

```
find /path/to/directory -type f -mtime +30
```

Explanation

- -type f → Files only
- -mtime +30 → Modified more than 30 days ago

Delete Old Files

```
find /var/log -type f -mtime +30 -delete
```

Q. Monitor logs in real time

```
tail -f /var/log/messages
```

```
monitor multiple logs: tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

Using journalctl: journalctl -f

For Specific Service: journalctl -u nginx -f

Q. Kill all Java processes

```
pkill java
```

```
killall java
```

Verify Processes Before Killing: ps -ef | grep java

Force Kill: pkill -9 java

Q. Check open ports

```
ss -tulnp
```

```
netstat -tulnp
```

Check specific Port: ss -tulnp | grep 8080

Option	Meaning
t	TCP
u	UDP
l	Listening
n	Numeric
p	Process

Q. Find top memory-consuming processes

```
ps aux --sort=-%mem | head
```

top and then press M(to sort by memory.)

Interactive and easier to read: htop

Other Questions:

Q. Find files larger than 1 GB

```
find / -type f -size +1G 2>/dev/null
```

Q. Check disk usage

```
df -h
```

Q. Check directory size

```
du -sh /var/log
```

Q. Find process using a port

```
lsof -i :8080
```

Q. Check current logged-in users

Who

Q. Monitor CPU usage in real time

```
mpstat 1
```

Windows Administration

1. Difference between Windows Server and Linux Server

Windows Server is mainly preferred in enterprise environments using Microsoft technologies such as Active Directory, IIS, and .NET applications. Linux servers are widely used for open-source applications, container platforms, and cloud-native environments because of their flexibility, performance, and cost efficiency.

Windows Server	Linux Server
GUI-based administration is common	Mostly CLI-based administration
Proprietary OS by Microsoft	Open-source OS
Uses Active Directory integration	Uses LDAP, Samba, Kerberos, etc.
Easier for Microsoft ecosystem	Preferred for open-source workloads
Licensing cost involved	Mostly free or low-cost
Uses PowerShell	Uses Bash/Shell scripting
Common for .NET, IIS, MS SQL	Common for Apache, Nginx, MySQL
Patching through Windows Update/WSUS	Uses package managers like yum, apt
Higher resource usage generally	Lightweight and highly customizable

2. What is Active Directory?

Active Directory (AD) is Microsoft's directory service used for centralized management of users, computers, groups, policies, and authentication inside a Windows domain environment.

Key Components:

- Domain
- Domain Controller (DC)
- Organizational Unit (OU)
- Users and Groups
- Group Policies
- Forest and Tree

Uses:

- Centralized authentication
- Authorization
- User management
- Policy enforcement
- Single Sign-On (SSO)

3. Explain Group Policy

Group Policy is a feature in Windows Server that allows administrators to centrally manage and configure operating systems, users, and computer settings in an Active Directory environment.

Types:

- Computer Configuration
- User Configuration

Common Use Cases:

- Password policies
- Disable USB ports
- Software installation
- Desktop wallpaper restrictions
- Security configurations

Commands:

`gpupdate /force`

`gpresult /r`

4. What is IIS?

IIS (Internet Information Services) is Microsoft's web server used to host websites, web applications, APIs, and services on Windows servers.

Supports:

- ASP.NET
- .NET Core
- PHP
- Static websites
- FTP
- HTTPS

Important Components:

- Application Pools
- Websites
- Bindings
- Virtual Directories

Common IIS Commands: iisreset

IIS Logs Location: `C:\inetpub\logs\LogFiles`

5. How do you manage Windows services?

Windows services are background processes managed through Services console, PowerShell, or SC commands. Administrators can start, stop, restart, or configure service startup behavior.

Windows services can be managed using:

- Services.msc
- Task Manager
- PowerShell
- CMD

PowerShell Commands:

Get-Service

Start-Service Spooler

Stop-Service Spooler

Restart-Service Spooler

Set-Service -Name Spooler -StartupType Automatic

6. Explain RDP troubleshooting

RDP (Remote Desktop Protocol) troubleshooting involves checking connectivity, services, firewall rules, and permissions.

Troubleshooting Steps:

Step 1: Verify Network Connectivity

ping <server-ip>

telnet <server-ip> 3389

Step 2: Check RDP Service

Get-Service TermService

Step 3: Verify Firewall

Ensure port 3389 is open.

netsh advfirewall firewall show rule name=all | findstr 3389

Step 4: Verify RDP Enabled

SystemPropertiesRemote

Step 5: Check User Permissions

User should belong to:

Remote Desktop Users

Step 6: Check Event Logs

Use Event Viewer:

Applications and Services Logs

→ Microsoft

→ Windows

→ TerminalServices

Common Issues:

- Network failure
- Firewall blocking
- Expired password
- Licensing issues
- NLA authentication failure
- DNS issues

7. What is Windows Event Viewer?

Event Viewer is a Windows utility used to view logs related to:

- System events
- Application events
- Security logs
- Setup logs

Main Log Categories:

Log Type	Purpose
Application	Application-related logs
System	OS and driver logs
Security	Login/audit logs
Setup	Installation logs

8. Explain Windows Performance Monitor

Performance Monitor (PerfMon) is a Windows tool used to monitor real-time system performance and resource utilization.

Open: perfmon

Counter	Purpose
CPU % Processor Time	CPU usage
Memory Available MBytes	Free memory
Disk Queue Length	Disk bottleneck
Network Interface	Network usage

Use Cases:

- Performance bottleneck analysis
- Capacity planning
- Troubleshooting server slowness
- Monitoring application performance

Interview Answer:

Performance Monitor helps track system health using counters related to CPU, memory, disk, and network performance.

9. How do you troubleshoot high CPU in Windows?

Step 1: Check CPU Usage

Use: taskmgr or Get-Process | Sort CPU -Descending

Step 2: Identify High CPU Process

Find:

- Application
- Service
- Background task

Step 3: Check Service Dependency: tasklist /svc

Step 4: Analyze Performance Monitor

Monitor:

- Processor Queue Length
- % Processor Time

Step 5: Check Event Viewer

Look for:

- Application crashes
- Driver issues
- Service failures

Step 6: Malware Scan

High CPU can also occur because of malicious processes.

Step 7: Verify Updates

Pending Windows updates or drivers can cause CPU spikes.

Common Causes:

- Memory leak
- Infinite loops
- Antivirus scans
- Windows updates
- Driver issues

10. Explain PowerShell remoting

PowerShell Remoting allows administrators to execute commands remotely on Windows machines using WinRM (Windows Remote Management).

- **Enable Remoting:** `Enable-PSRemoting -Force`
- **Remote Command Execution:** `Invoke-Command -ComputerName Server1 -ScriptBlock { Get-Service }`
- **Interactive Remote Session:** `Enter-PSSession -ComputerName Server1`
- **Multiple Server Execution:** `Invoke-Command -ComputerName Server1,Server2 -ScriptBlock { hostname }`

Requirements:

- WinRM enabled
- Firewall rules configured
- Proper authentication/permissions

Authentication Methods:

- Kerberos
- NTLM
- CredSSP
- Certificate-based

Windows Troubleshooting Questions

1. Windows Server is Slow After Patching — How Do You Troubleshoot?

After Windows patching, slowness can happen due to pending updates, driver incompatibility, high resource utilization, corrupted patches, indexing, antivirus scans, or services stuck after reboot.

Step-by-Step Troubleshooting

A. Verify System Resource Usage

Check CPU, Memory, Disk, and Network utilization.

Use:

- Task Manager
- Resource Monitor (resmon)
- Performance Monitor (perfmon)

PowerShell: `Get-Process | Sort CPU -Descending`

Check:

- High CPU by TrustedInstaller.exe
- TiWorker.exe
- Windows Update service
- Antivirus scans
- Excessive paging

B. Check Windows Update Status

Sometimes updates remain incomplete.

Commands: `Get-HotFix`

Check pending reboot: `Get-ItemProperty`

`"HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\RebootPending"`

C. Review Event Viewer

Look for:

- Update failures
- Driver errors
- Service failures

- Disk issues

Important Logs:

- System
- Application
- Setup

Event IDs:

- 20
- 25
- 1001
- 6008

D. Check Disk Performance

High disk latency after patching is common.

Use: perfmon

Monitor:

- Avg Disk Queue Length
- Disk Transfers/sec
- Disk Latency

If queue length remains high:

- Storage bottleneck
- Driver issue
- Antivirus scanning

E. Validate Services

Some services may hang after updates.

Check: `Get-Service | Where-Object {$_.Status -ne "Running"}`

F. Check Drivers

Patching can break drivers.

Open: `devmgmt.msc`

Look for:

- Warning icons
- Disabled drivers
- NIC/storage driver issues

G. Run System Integrity Checks

sfc /scannow

DISM /Online /Cleanup-Image /RestoreHealth

H. Roll Back Problematic Patch

If issue started immediately after KB installation: wusa /uninstall /kb:123456

2. IIS Website is Down — What Checks Will You Perform?

A. Verify IIS Services

Check: iisreset /status

Required services:

- W3SVC
- WAS

Start services: net start w3svc

B. Check Application Pool

Most common reason.

Open: inetmgr

Check:

- App Pool stopped
- Rapid-Fail Protection triggered
- Identity password expired
- .NET version mismatch

Restart: Restart-WebAppPool "AppPoolName"

C. Verify Website Binding

Check:

- Port conflict
- Wrong hostname
- SSL certificate expired

Command:

netstat -ano | findstr :80

netstat -ano | findstr :443

D. Test Website Locally

From server: curl <http://localhost>

If localhost works but external fails:

- Firewall
- DNS
- Load balancer
- NSG issue

E. Check IIS Logs

Location: C:\inetpub\logs\LogFiles

Common codes:

- 500 → application issue
- 503 → app pool unavailable
- 401 → authentication
- 404 → missing content

F. Review Event Viewer

Check:

- WAS errors
- ASP.NET crashes
- DLL failures

G. Check Disk Space

IIS may stop functioning when disk is full.

Check: nslookup domain.com

3. Users Cannot Authenticate with AD — What Could Be the Reasons?

Common Causes

A. DNS Problems

AD heavily depends on DNS.

Check: nslookup domain.com

Verify:

- Correct DC resolution
- SRV records

B. Domain Controller Unreachable

Check:

ping dc01

nltest /dsgetdc:domain.com

C. Time Synchronization Issue

Kerberos fails if time difference >5 minutes.

Check: w32tm /query /status

Sync: w32tm /resync

D. Account Lockout

Check:

- Account locked
- Password expired
- Disabled account

Use: Get-ADUser username -Properties LockedOut

E. Replication Failure

If multiple DCs exist: repadmin /replsummary

F. Secure Channel Broken

Repair: Test-ComputerSecureChannel -Repair

4. Windows Service Keeps Stopping Automatically — How Do You Debug?

A. Check Event Viewer

Look for:

- Service crashes
- Dependency failures
- Timeout errors

Important Event IDs:

- 7031
- 7034
- 7000

B. Check Service Dependencies

sc qc ServiceName

If dependency service fails, target service also stops.

C. Verify Service Account

Check:

- Password expired
- Account locked
- Missing permissions

D. Enable Service Recovery

services.msc

Recovery tab:

- Restart service
- Run program
- Reboot server

E. Analyze Crash Dumps

Use:

- ProcDump
- WinDbg

Generate dump: procdump -e -ma service.exe

F. Check Resource Exhaustion

Low memory or CPU spikes may terminate services.

Check:

- Memory usage
- Handle count
- Thread count

5. High Disk Queue Length Issue — What Does It Indicate?

High Disk Queue Length means disk requests are waiting because storage cannot process I/O fast enough.

Indicates:

- Storage bottleneck
- Slow disk subsystem
- High read/write operations
- SAN latency
- Antivirus scans
- SQL/IIS heavy load

Important Metrics

Healthy Range

Rule of thumb: Queue Length $\leq$ 2 per disk spindle

Check Using PerfMon

Counters:

- Avg Disk Queue Length
- Disk sec/Read
- Disk sec/Write

Common Causes

- Insufficient IOPS
- Fragmentation
- Backup jobs
- Paging due to low RAM
- Storage driver issues

Troubleshooting

- Identify top I/O processes: resmon
- Check: Get-Process

Move workload to faster storage

Add RAM

Optimize SQL/IIS logs

6. How Do You Troubleshoot Memory Leaks in Windows?

Memory leak occurs when process memory continuously increases and never releases.

A. Identify Process

Use:

- Task Manager
- Resource Monitor
- Performance Monitor

Counters:

- Private Bytes
- Working Set
- Handle Count

B. Monitor Over Time

Use: perfmon

Track:

- Process → Private Bytes
- Memory → Available MBytes

C. Use Process Explorer

Look for:

- Increasing handles
- Increasing threads
- Unreleased memory

D. Capture Memory Dump

Use: procdump -ma PID

Analyze with:

- WinDbg
- DebugDiag

E. Common Causes

- IIS worker process leaks
- .NET unmanaged memory
- Driver leaks
- Antivirus
- Third-party agents

F. IIS Leak Example

Recycle app pool: iisreset

Configure:

- Private Memory Limit
- Scheduled recycling

7. Event Viewer Shows Repeated Critical Errors — What Next?

A. Identify Error Pattern

Check:

- Event ID
- Source
- Frequency
- Timestamp correlation

B. Categorize Issue

Possible categories:

- Disk
- Driver
- Service
- Hardware
- Security
- Application crash

C. Correlate with System Changes

Ask:

- Was patching done?
- Driver updated?
- Storage changes?
- New software installed?

D. Check Reliability Monitor

Open: perfmon /rel

Provides crash timeline.

E. Enable Advanced Logging

Use:

- ProcMon
- Debug logs
- Application tracing

F. Check Hardware

Critical events may indicate:

- Bad RAM
- Failing disk
- PSU issue

Run:

chkdsk

mdsched.exe

8. RDP Connection Failing Intermittently — How Do You Investigate?

A. Verify Network Connectivity

Check:

ping server

tracert server

Test port: Test-NetConnection server -Port 3389

B. Verify RDP Service

Get-Service TermService

Restart: Restart-Service TermService

C. Check Event Viewer

Logs:

Applications and Services Logs

→ Microsoft

→ Windows

→ TerminalServices

D. Check Session Limits

Possible causes:

- Max sessions reached
- Disconnected sessions consuming slots

Command: query session

E. Firewall and NSG

Verify:

- Port 3389 open
- NSG rules
- VPN stability

F. DNS Issues

Intermittent failures may occur due to:

- DNS flapping
- Multiple IPs
- Wrong host records

G. Resource Exhaustion

Low memory or CPU may terminate RDP sessions.

Check:

- CPU spikes
- RAM pressure

9. How Do You Check Failed Login Attempts?

A. Event Viewer Security Logs

Open: eventvwr.msc

Path: Windows Logs → Security

B. Important Event IDs

Failed Login: 4625

Account Lockout: 4740

C. PowerShell Query

```
Get-WinEvent -FilterHashtable @{LogName='Security';ID=4625}
```

D. Check Failure Details

Look for:

- Username
- Source IP
- Logon type
- Failure reason

E. Common Failure Reasons

- Bad password
- Locked account
- Expired password
- Time synchronization issue
- Brute force attack

10. Explain BSOD Troubleshooting Steps

BSOD = Blue Screen of Death caused by kernel-level failures.

Step-by-Step BSOD Troubleshooting

A. Identify Stop Code

Examples:

- IRQL_NOT_LESS_OR_EQUAL
- PAGE_FAULT_IN_NONPAGED_AREA
- CRITICAL_PROCESS_DIED

B. Analyze Dump Files

Location: C:\Windows\Minidump

Use:

- WinDbg
- BlueScreenView

Command in WinDbg: !analyze -v

C. Check Event Viewer

Look for:

- Kernel-Power
- BugCheck events

Event IDs:

- 41
- 1001

D. Check Drivers

Most BSODs are driver related.

Use: driverquery

Update:

- NIC drivers
- Storage drivers
- GPU drivers

E. Test Memory

Run: mdsched.exe

Or use MemTest86.

F. Check Disk Health

chkdsk /f /r

G. Run System Integrity Checks

sfc /scannow

DISM /Online /Cleanup-Image /RestoreHealth

H. Boot in Safe Mode

If server crashes repeatedly:

- Boot Safe Mode
- Remove problematic drivers/patches

I. Hardware Investigation

Possible causes:

- Faulty RAM
- Overheating
- Disk failure
- RAID controller issue

